

SKILL-22: PROCESS SYNCHRONIZATION AND CHILD PROCESS MONITORING USING waitpid()

Name: Simran Jangid

Roll No: 2520030166

Sec: 8

PART 1: PROCESS SYNCHRONIZATION AND CHILD PROCESS MONITORING USING waitpid()

Aim

To write a C program that creates multiple child processes using fork(), monitors their execution using waitpid(), synchronizes their completion, and collects their exit status.

Theory

The fork() system call creates a new child process from the parent process.

The waitpid() system call is used by the parent process to wait for a **specific child process** to terminate.

When a child process completes its execution, the parent can use waitpid() to collect its termination status. This provides proper synchronization between the parent and child processes.

waitpid() is more flexible than wait() because it allows the parent to specify which child process it wants to wait for.

Syntax

```
waitpid(pid, &status, options);
```

Parameters

- pid – Specifies the child process to wait for.
- &status – Stores the termination status of the child.
- options – Specifies the waiting behavior.

For normal blocking execution:

```
waitpid(pid, &status, 0);
```

Objectives

1. Create multiple child processes.
2. Obtain the PID of each child.
3. Execute different tasks in child processes.
4. Monitor child completion using waitpid().
5. Wait for a specific child process.
6. Collect the exit status of each child.
7. Synchronize parent and child processes.
8. Prevent child processes from remaining as zombies.

System Calls / Functions Used

Function	Purpose
fork()	Creates a new child process
waitpid()	Waits for a specific child process

getpid()	Returns the current process ID
sleep()	Delays process execution
printf()	Displays information
WEXITSTATUS()	Retrieves the exit status of a terminated child

C Code

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main()
{
    pid_t p1, p2;
    int status;

    p1 = fork();

    if (p1 == 0)
    {
        printf("Child 1: PID = %d\n", getpid());
        sleep(2);
        printf("Child 1 completed\n");
        return 10;
    }

    p2 = fork();

    if (p2 == 0)
    {
        printf("Child 2: PID = %d\n", getpid());
        sleep(1);
        printf("Child 2 completed\n");
        return 20;
    }
}
```

```
printf("Parent: PID = %d\n", getpid());

waitpid(p1, &status, 0);
printf("waitpid(): Child 1 status = %d\n",
      WEXITSTATUS(status));

waitpid(p2, &status, 0);
printf("waitpid(): Child 2 status = %d\n",
      WEXITSTATUS(status));

printf("Parent process completed\n");

return 0;
}
```

Explanation of Code

1. Header Files

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

These provide functions required for input/output, process creation, and process synchronization.

2. Creating Child 1

```
p1 = fork();
```

This creates the first child process.

If `p1 == 0`, the program is executing inside Child 1.

3. Child 1 Execution

```
sleep(2);  
return 10;
```

Child 1 waits for 2 seconds and terminates with exit status 10.

4. Creating Child 2

```
p2 = fork();
```

The parent creates the second child process.

Child 2 waits for 1 second and terminates with exit status 20.

5. Parent Process

The parent displays its PID and waits for its children.

6. waitpid() for Child 1

```
waitpid(p1, &status, 0);
```

The parent waits specifically for Child 1.

7. Collecting Status

```
WEXITSTATUS(status)
```

This extracts the exit status returned by the child.

8. waitpid() for Child 2

```
waitpid(p2, &status, 0);
```

The parent then waits for Child 2.

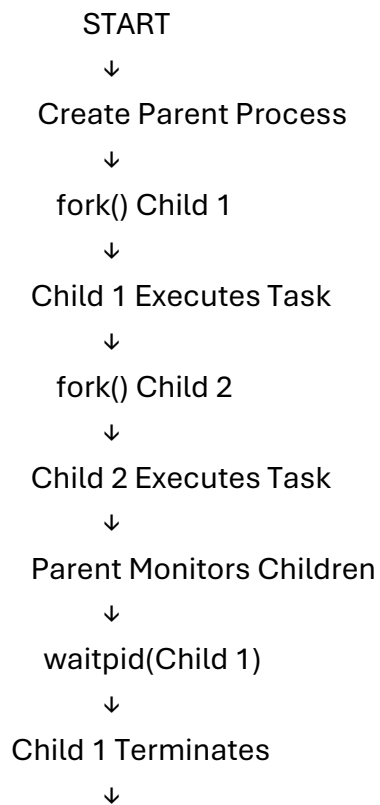
Compile

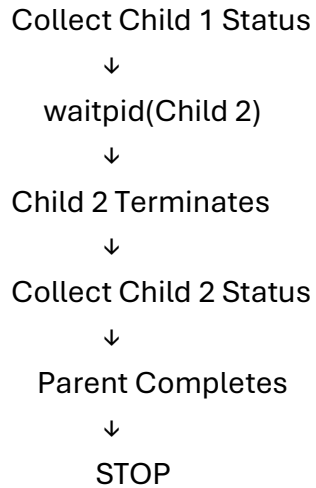
```
gcc monitor.c -o monitor
```

Run

```
./monitor
```

Flow Chart





Sample Output

Parent: PID = 4500
Child 1: PID = 4501
Child 2: PID = 4502
Child 2 completed
Child 1 completed
waitpid(): Child 1 status = 10
waitpid(): Child 2 status = 20
Parent process completed

Note: PID values and the exact order of child output may vary during execution.

Observation

Function / Operation	Behavior
<code>fork()</code>	Creates a new child process
<code>getpid()</code>	Displays the process ID
<code>sleep()</code>	Delays child execution
<code>waitpid()</code>	Waits for a particular child
<code>WEXITSTATUS()</code>	Gets the child's exit status
Parent process	Monitors child processes
Child process	Executes independently
Synchronization	Parent waits for child completion

Comparison of wait() and waitpid()

Feature	wait()	waitpid()
Child selection	Any child	Specific child
PID selection	Not specified	PID can be specified
Control	Less control	More control
Synchronization	Yes	Yes
Status collection	Yes	Yes
Use	When any child can be waited for	When a particular child must be monitored

Expected Behavior

- The parent creates two child processes.
- Child 1 and Child 2 execute independently.
- Child 2 completes earlier because it sleeps for only 1 second.
- The parent specifically waits for Child 1 using waitpid().
- After Child 1 completes, its exit status is collected.
- The parent then waits for Child 2.
- After both children are handled, the parent process terminates.

Important Points

1. fork() creates a new process.
2. Each child has a unique PID.
3. waitpid() allows specific child monitoring.

4. The parent can collect the child's exit status.
5. WEXITSTATUS() gives the value returned by the child.
6. waitpid() provides process synchronization.
7. Proper use of waitpid() helps prevent zombie processes.
8. The parent continues after the required child process terminates.

OUTPUT

```
[gcc monitor.c -o monitor && ./monitor  
Parent: PID = 4968  
Child 1: PID = 4969  
Child 2: PID = 4970  
Child 2 completed  
Child 1 completed  
waitpid(): Child 1 status = 10  
waitpid(): Child 2 status = 20  
Parent process completed  
7rainreddy7@Gunapatis-MacBook-Air ~ %
```

```
Parent: PID = 4500  
Child 1: PID = 4501  
Child 2: PID = 4502  
Child 2 completed  
Child 1 completed  
waitpid(): Child 1 status = 10  
waitpid(): Child 2 status = 20  
Parent process completed
```

Result

The C program successfully created multiple child processes using `fork()` and monitored them using `waitpid()`. The parent process synchronized with the child processes, waited for specific child PIDs, and successfully collected their termination statuses. Thus, process synchronization and child process monitoring using `waitpid()` were successfully demonstrated.